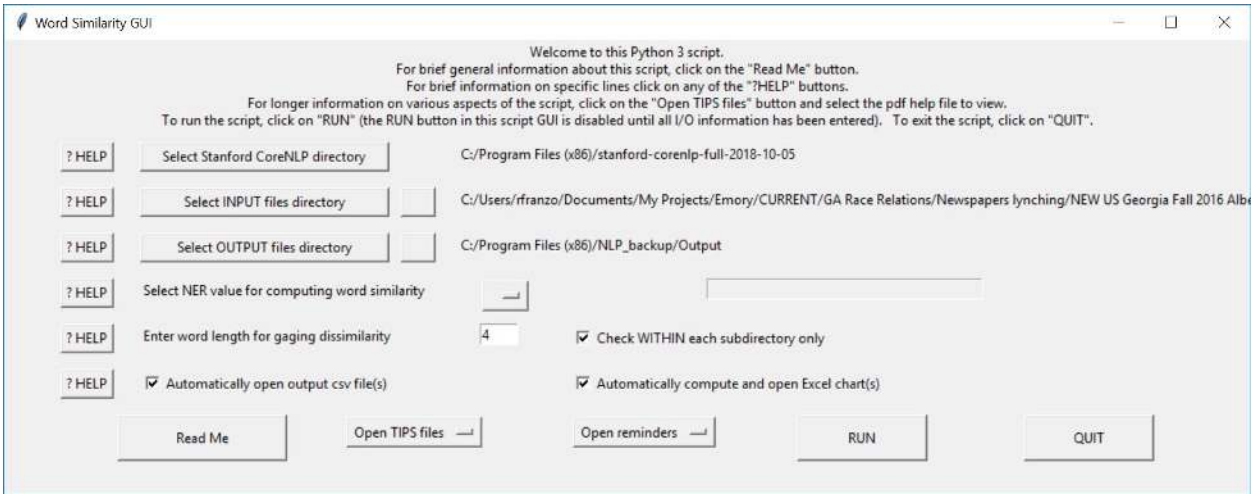
Word Similarity (Levenshtein Distance)

Table of contents

Proper noun spelling checker 1
NER (Named Entity Recognition) annotator 1
Levenshtein's word/edit distance 2
INPUT 2
OUTPUT 4
References 4

Proper noun spelling checker

The algorithm uses two different sets of tools (a NER, Named Entity Recognition, annotator and Levenshtein's word/edit distance) to check for possible misspellings in proper names, i.e., names that would be tagged as NNP or NNPS in the POSTAG values of the CoNLL table.



NER (Named Entity Recognition) annotator

The algorithm takes as input the NER values of POSTAG (Part of Speech tag) proper names (NNP and NNPS, proper noun singular and plural), i.e., City, Country, State-Or-Province, Location, Organization, Person, as computed by the NLP package you selected in the NLP Suite setup (Stanford CoreNLP, Stanza, or spaCy). You are asked for the Stanford CoreNLP directory only when CoreNLP is the package you selected.

Please, be aware that the three finest-grained location tags, City, Country and State-Or-Province, are computed by Stanford CoreNLP alone. Stanza and spaCy use a model in which a city, a state and a country are all a single Location entity. If you select one of those three tags while Stanza or spaCy is your configured package, the NLP Suite will tell you so and continue with the remaining tags, rather than return an empty result that might be mistaken for an absence of misspellings.

Named entity results differ between packages more generally, because each package uses a different model: a newspaper title such as The Atlanta Constitution is tagged Organization by Stanford CoreNLP but Law by Stanza. If you switch package in mid-project, re-check a few documents before comparing new results against old ones.

Levenshtein's word/edit distance

The Levenshtein's word or edit distance is defined as the smallest number of insertions, deletions, and substitutions required to change one string or tree into another. (2) A $\Theta(m \times n)$ algorithm to compute the distance between strings, where m and n are the lengths of the strings (Levenshtein 1966).
The NLP Suite computes the edit distance directly, in its string_similarity_util module, rather than through an external library (for a long list of implementations, in different programming languages, see the webpage at the US National Institute of Standards, NIS, https://xlinux.nist.gov/dads/HTML/Levenshtein.html). Earlier releases called the fuzzywuzzy package, which computes a true Levenshtein distance only when the optional python-Levenshtein C library is installed and otherwise falls back, with nothing more than a warning, to a different algorithm altogether (the Ratcliff/Obershelp matcher of the difflib library). The same corpus could therefore be scored differently on two installations, and on any machine without that C library the tool was not computing a Levenshtein distance at all. Computing the distance inside the Suite removes the dependency: results are now identical on every installation.
The distance is scaled to a similarity score running from 0 to 100, where 100 means identical. The comparison ignores case and word order, so that COBB and Cobb, and Jim Cobb and Cobb, Jim, are recognized as the same name; when several candidate spellings pass the threshold, it is the closest one that is reported.
Be aware that a single similarity threshold behaves differently on short and on long words. On a short surname one edit already costs a large share of the score (Lee and Loe score 67), so a genuine typo may fall below the threshold; two genuinely distinct short names, meanwhile, may sit above it (Macon and Bacon score 80). On longer strings the opposite holds: Atlanta Constitution and Atlanta Constitutional score 95 in spite of two edits. Inspect the suggested variants, therefore, rather than accepting them wholesale.

INPUT

The algorithm can either process all the txt files in a directory (Fig. 1), including its subdirectories, computing for each file the user-selected NER values and comparing values with the NER values of every other file.

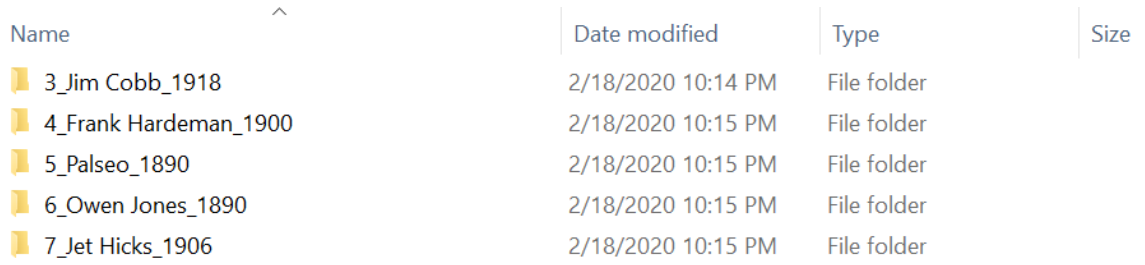
Name	Date modified	Type	Size
Charlotte Daily Observer_09-23-1906_1_1.txt	2/8/2020 2:42 PM	TXT File	6 KB
Chicago Daily Tribune_09-23-1906_1_5.txt	2/8/2020 2:42 PM	TXT File	7 KB
Chicago Daily Tribune_09-24-1906_8_4.txt	2/8/2020 2:42 PM	TXT File	3 KB
Daily Press_09-23-1906_1_1.txt	2/8/2020 2:42 PM	TXT File	5 KB
Dallas Morning News_09-23-1906_2_1.txt	2/8/2020 2:42 PM	TXT File	1 KB
Dallas Morning News_09-23-1906_2_1_2.txt	2/8/2020 2:42 PM	TXT File	1 KB
Dallas Morning News_09-23-1906_2_1_3.txt	2/8/2020 2:42 PM	TXT File	1 KB
Dallas Morning News_09-23-1906_2_1_4.txt	2/8/2020 2:42 PM	TXT File	1 KB
Dallas Morning News_09-23-1906_2_1_5.txt	2/8/2020 2:42 PM	TXT File	1 KB
Dallas Morning News_09-23-1906_2_1_6.txt	2/8/2020 2:42 PM	TXT File	1 KB
Los Angeles Time_09-23-1906_2_4.txt	2/8/2020 2:42 PM	TXT File	4 KB
Los Angeles Times_09-24-1906_1_2.txt	2/8/2020 2:42 PM	TXT File	8 KB
Los Angeles Times_09-24-1906_6_1.txt	2/8/2020 2:42 PM	TXT File	1 KB
New York Times_09-23-1906_1_1.txt	2/8/2020 2:42 PM	TXT File	9 KB
New York Times_9-24-1906_2_5.txt	2/8/2020 2:42 PM	TXT File	11 KB

Figure 1 – Document list in a directory

The algorithm can also limit the computation and computing of user-selected NER values to the files *within* subdirectories. Thus, if a user has organized the documents (e.g., newspaper articles) that describe specific events in subdirectories, each subdirectory bearing the ID of the event in a database, a specific name, or a combination of ID and/or name, the user can restrict the NER computations and comparisons to *within* each subdirectory (see Fig. 2).

Name	Date modified	Type	Size
3	2/18/2020 10:14 PM	File folder	
4	2/18/2020 10:15 PM	File folder	
5	2/18/2020 10:15 PM	File folder	
6	2/18/2020 10:15 PM	File folder	
7	2/18/2020 10:15 PM	File folder	

Name	Date modified	Type	Size
3_Jim Cobb	2/18/2020 10:14 PM	File folder	
4_Frank Hardeman	2/18/2020 10:15 PM	File folder	
5_Palseo	2/18/2020 10:15 PM	File folder	
6_Owen Jones	2/18/2020 10:15 PM	File folder	
7_Jet Hicks	2/18/2020 10:15 PM	File folder	



Name	Date modified	Type	Size
3_Jim Cobb_1918	2/18/2020 10:14 PM	File folder	
4_Frank Hardeman_1900	2/18/2020 10:15 PM	File folder	
5_Palseo_1890	2/18/2020 10:15 PM	File folder	
6_Owen Jones_1890	2/18/2020 10:15 PM	File folder	
7_Jet Hicks_1906	2/18/2020 10:15 PM	File folder	

Figure 2 – Event subdirectories structure

OUTPUT

References

Levenshtein, Vladimir I. 1966. “Binary Codes Capable of Correcting Deletions, Insertions, and Reversals,” Doklady Akademii Nauk SSSR, 163(4):845-848, 1965 (Russian). English translation in *Soviet Physics Doklady*, 10(8):707-710, 1966. (Doklady is Russian for "Report". Sometimes transliterated in English as Doclady or Dokladi.)